PROJECT REFLECTION

Project: Cloud Command Parser

Name: Ayush Singh

Roll No.: 230101008

GitHub: github.com/ayush0singh/cloud-command-parser

1. What I Learned

This mini project helped me understand how a small language model can convert normal English commands into a format that a computer can process. The parser gives a JSON output with four fields: intent, date, time and place. This fixed format makes it easier for other applications to use the result.

I also learned the basic steps of making a machine-learning application, such as preparing examples, labelling and cleaning data, training or evaluating a model, and providing the result through an API.

One new concept was knowledge distillation. In this method, a larger teacher model provides outputs that help train a smaller student model. The aim is to make the model lighter while keeping useful performance.

I understood that a cloud application needs more than just a model. Storage, containers, deployment, networking, testing and performance checks are also important. Object storage keeps datasets and files, Docker packages the application, Kubernetes manages containers, and an API connects the parser to other programs.

2. Skills and Tools

During this assignment, I revised Python and worked with project scripts and configuration files. I also learned the purpose of FastAPI, Docker, Kubernetes manifests, object storage and basic network access rules. This helped me understand how different parts work together in one application.

PROJECT REFLECTION — CONTINUED

3. Challenges Faced

The first challenge was understanding the full workflow because the assignment combines machine learning and cloud infrastructure. At first, data labelling, model training, API deployment, storage and networking seemed like separate tasks. Dividing the work into smaller parts made it easier.

Another challenge was getting correct structured output from the language model. Sometimes a model may add extra text, miss a field or return invalid formatting. Since the application expects JSON, cleaning the data and validating the output are very important.

Setting up the infrastructure was also difficult. Docker and Kubernetes need correct configuration, and writing a configuration file does not guarantee that deployment will work. I also learned that subnet and NACL rules must be checked carefully because wrong rules can block or allow unwanted traffic.

4. How I Approached the Challenges

I divided the project into smaller components and checked the code and configuration files one by one. I used small tests to find basic errors before moving to more complex tasks. For networking, sample cases helped me check whether the access rules gave the expected allow or deny result.

I also understood the difference between writing code and proving that a feature works. For example, creating a Kubernetes manifest is not the same as successfully deploying and testing the service. In the same way, a training script alone does not prove that the student model has finished training.

5. Findings

My main finding is that a useful command parser needs more than a model that gives sensible text. It needs correct labels, valid JSON, input and output checks, and a deployment setup that is tested. Keeping commands, software versions, configuration and results also makes the work easier to repeat.

PROJECT REFLECTION — FINAL PAGE

6. Limitations and Improvements

The project is not fully complete in every part. Student-model training, deployment on the local Kubernetes cluster and load testing still need proper end-to-end execution and evidence. I should report these activities as completed only when actual logs and outputs are available.

If I continue this project, I will complete the data generation and training process first and test the student model on a separate validation set. After that, I will deploy the API on the local cluster and measure its availability, latency and throughput.

I would also improve error handling so that unclear commands or invalid model responses produce proper error messages. The documentation can include setup commands, dependency versions, sample inputs and outputs, test results and deployment steps.

In a future version, I would add more training examples, improve the handling of ambiguous commands and connect each requirement with its related test or implementation.

7. Overall Reflection

Overall, this assignment gave me practical experience in building and packaging a small AI-based application. It helped me connect machine learning, software development, cloud computing and computer networking.

I learned that a good technical project should not only contain code, but should also show evidence that important features have been tested. This assignment also helped me identify the topics I understand and the areas where I need more hands-on practice.

Student: Ayush Singh

Roll No.: 230101008

GitHub: github.com/ayush0singh/cloud-command-parser